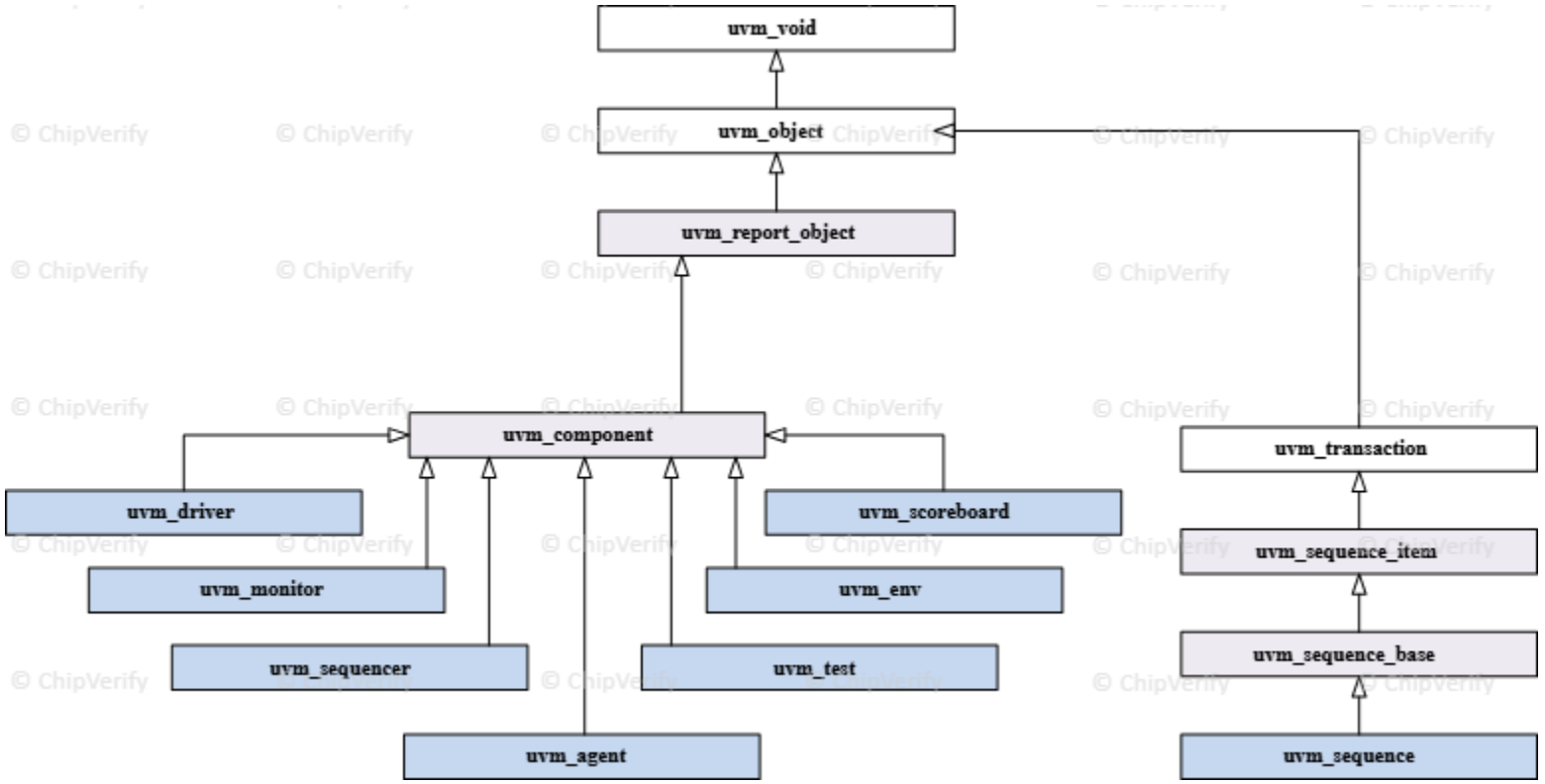
UVM Review

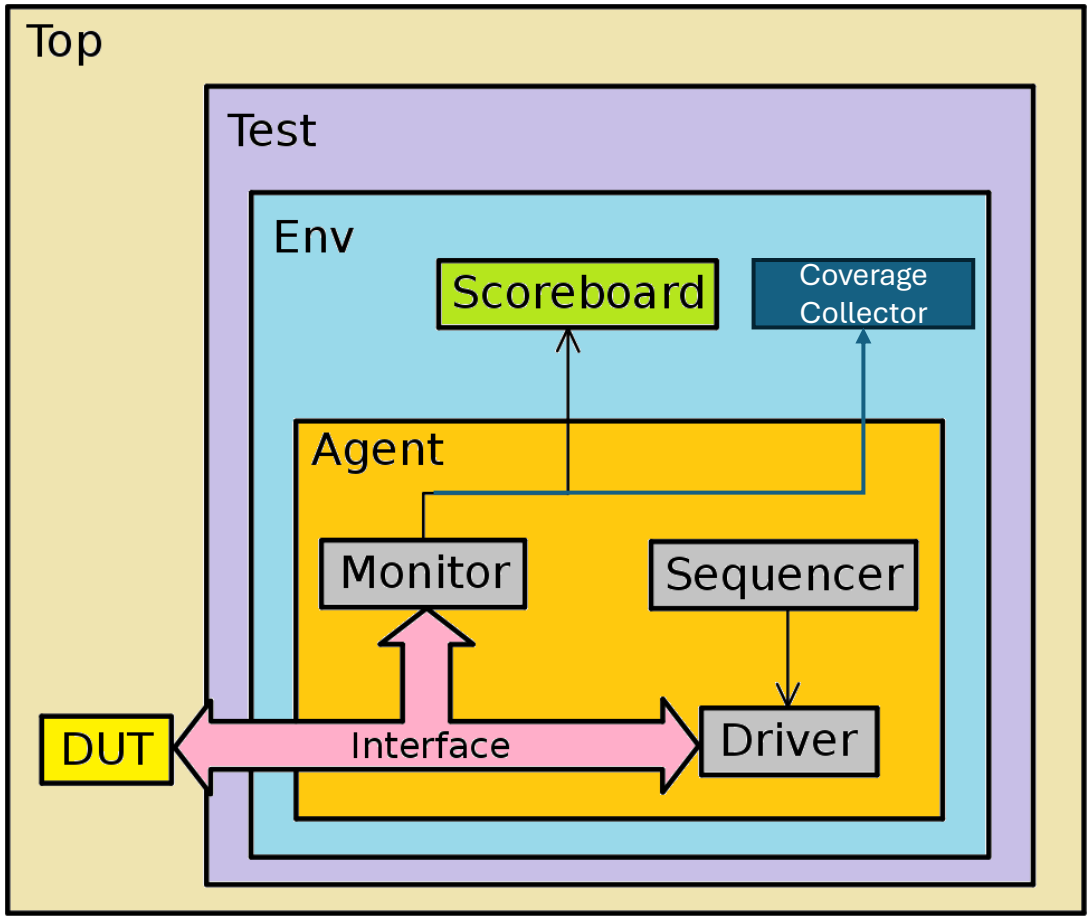
Objects vs. Components

- Objects:
  - Created and destroyed during the simulation
  - Examples: Transactions
- Components:
  - Created once in the beginning of the simulation. Never destroyed.
  - Participate in phasing
  - Examples: Driver, Monitor, Agent
  - Active vs. Passive components

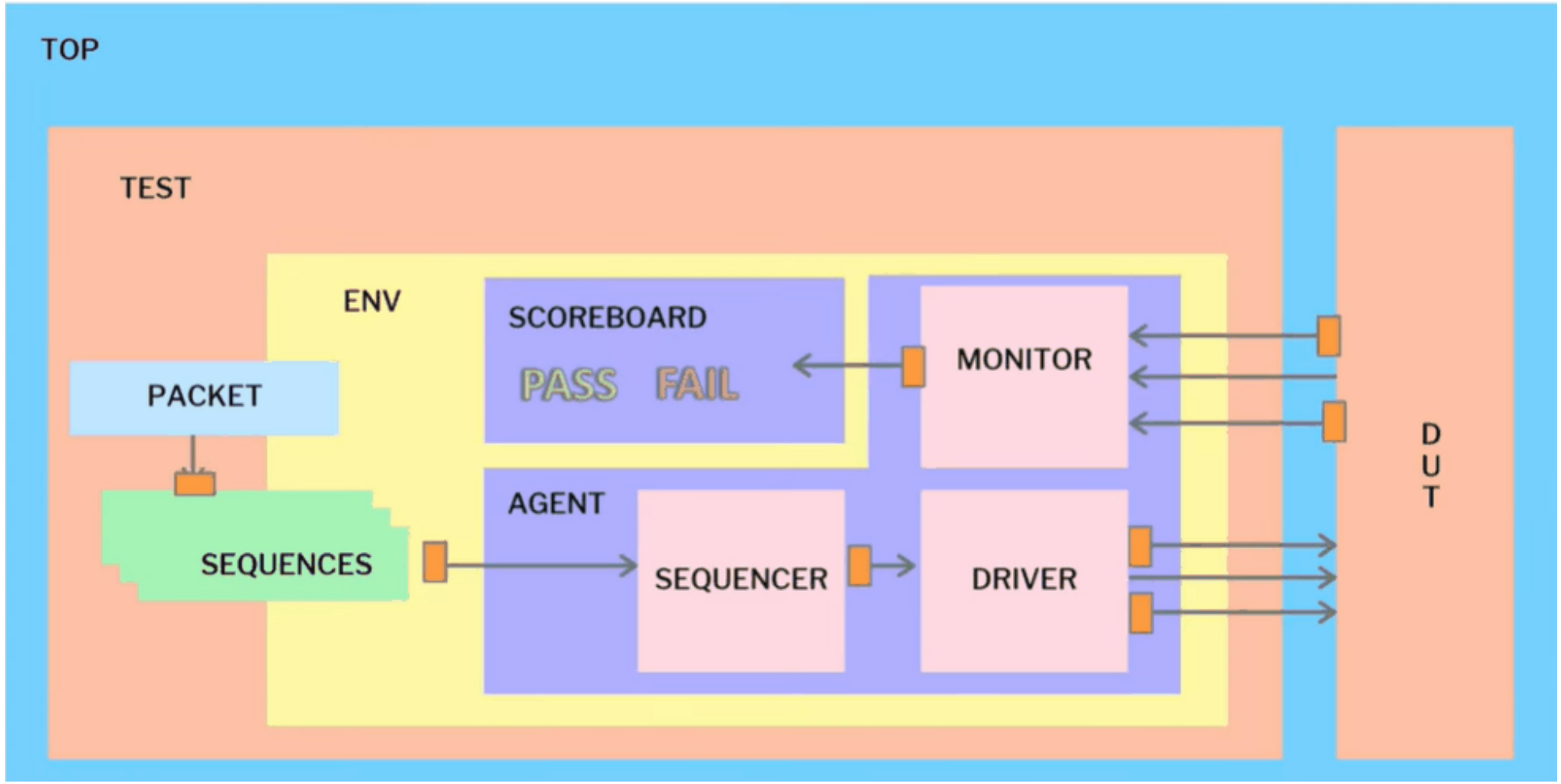
UVM Class Hierarchy



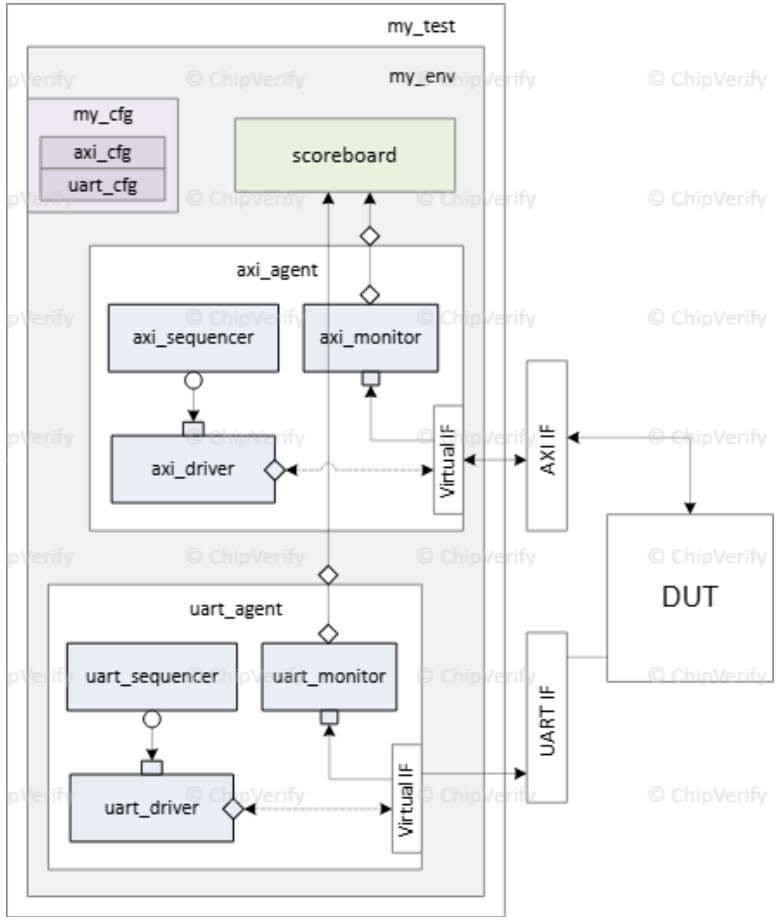
UVM Testbench Hierarchy



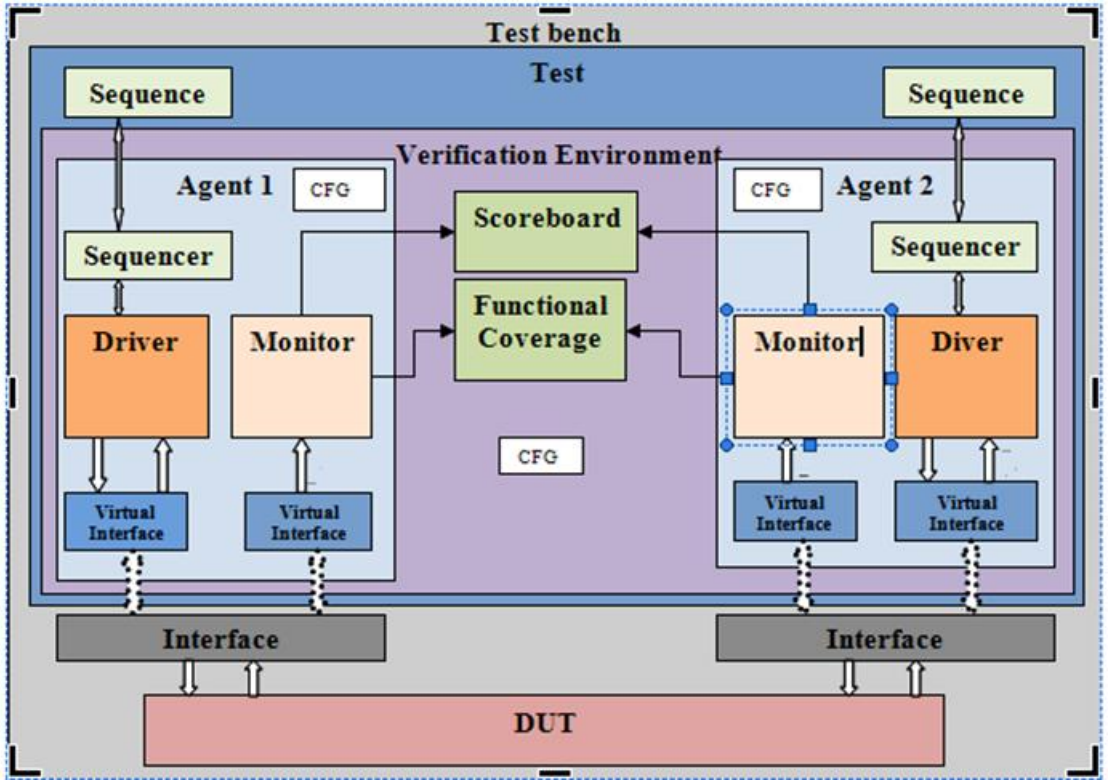
UVM Data Flow



UVM Testbench Hierarchy



UVM Testbench Hierarchy



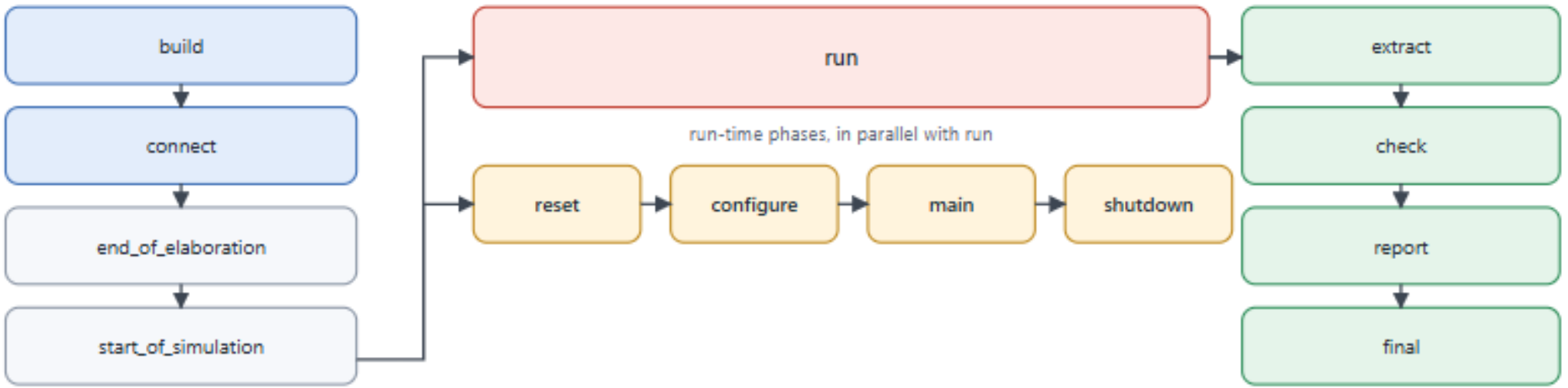
UVM Data Flow

- Very cool animation:
  - UVM Architecture — Interactive Animated Diagram | EcrioniX

Slide 9 omitted from this copy.

It reproduces a figure copyrighted by Joel D. Feldman (UVM 1),
whose slides are licensed for ASU CSE/CEN 598 coursework only.
See the original deck on the course site.

UVM Phasing



Only the run phase and the run-time phases consume simulation time; all the others are functions.

Nomenclature

- VIP
- Agent, BFM
- Driver, Monitor, Transactor
- Parent (tb hierarchy) vs. Super (class hierarchy)

UVM config db vs. resource db

- Config_db is built over resource_db
- Resource_db is the general underlying mechanism
- Config_db is more for uvm components (it’s hierarchy aware)

```
uvm_config_db#(int)::set(
    this,
    "env.agent",
    "num_transactions",
    100
);
```

```
int num_transactions;

uvm_config_db#(int)::get(
    this,
    "",
    "num_transactions",
    num_transactions
);
```

```
uvm_resource_db#(int)::set(
    "GLOBAL",
    "timeout",
    1000
);
```

```
int timeout;

uvm_resource_db#(int)::read_by_name(
    "GLOBAL",
    "timeout",
    timeout
);
```

Field macros

- Say you have a class and some members in it
- You need to define a function that can copy values of all members from one object to another
- You will need to define a function for that (say copy())

- Field macros provide these for free.
- And many other functions
- Example: .print(), .copy(), .compare(), etc.

Object utils vs. Component utils macros

- `uvm_object_utils(my_transaction)
- txn = my_transaction::type_id::create("txn");

- `uvm_component_utils(my_driver)
- driver = my_driver::type_id::create("driver", this);

UVM Factory

- Provides a way to swap objects/components without changing the testbench
- Example: create a error_injection_driver instead of a regular driver
- Enables test-specific customization
- Use ::type_id::create()
- More of an advanced concept; We won’t use the override feature. But use ::type_id::create() anyway.

Sending transactions

- Option 1:
  - `uvm_do()
- Option 2:
  - `uvm_create(req);
  - req.randomize();
  - `uvm_send(req);
- Option 3:
  - txn = transaction::type_id::create(“”)
  - start_item(txn)
  - txn.randomize()
  - finish_item(txn)

Port, export, imp

- Port -> I want to call/send
- Export -> Forwards
- Imp -> I provide an implementation

UVM Analysis Port vs Analysis Export vs Analysis Imp — DV Verification Hub

Ports vs. Analysis Ports

- Typical use of a port: Connect driver and sequencer
  - Blocking or non-blocking
  - .put(), .get(), etc.
- Typical use of analysis port: Connect monitor to a subscriber
  - Analysis port is a specific type of port, designed for broadcasting transactions typically from a monitor
  - A subscriber (coverage collector, scoreboard) will connect to an analysis_port and provide a .write()
  - Non-blocking
  - .write()

Needing to store transactions to do analysis

- uvm_analysis_port is a broadcast mechanism. When a monitor posts the transaction to the analysis, the corresponding .write() is called. There is no queue/storage.
- What is the consumer/subscriber needs to process transactions later (may need multiple transactions before it can process them).
- Use uvm_tlm_analysis_fifo
  - uvm_tlm_analysis_fifo #(transaction) myfifo;
  - monitor.ap.connect(myfifo.analysis_export);
  - myfifo.get(txn);

Needing more than one analysis port on a class

- Normally, a uvm_analysis_imp expects your component to implement a write method
- If you are connecting to two analysis ports, how do you distinguish?
- Use
  - `uvm_analysis_imp_decl(_before)
  - `uvm_analysis_imp_decl(_after)
- Now, you can have:
  - function void write_before(transaction t);
  - function void write_after(transaction t);
- And connections will be:
  - input_monitor.ap.connect(scoreboard.before_imp);
  - output_monitor.ap.connect(scoreboard.after_imp);

Common UVM debug methods

- Call .print() on any object
- Use `uvm_info, `uvm_warning, `uvm_error, `uvm_fatal
- Compare two objects
  - actual.compare(expected)
- Introspection functions
  - this.get_full_name, txn.get_type_name()
- Print UVM testbench topology
  - uvm_top.print_topology()

SV testbenches vs. UVM testbenches

- SV: You are on your own. Design classes the way you want. Limited reuse.
- UVM: Provides a standard setup/flow. High focus on reuse.

SV testbenches vs. UVM testbenches

SV

- Stimulus: Tasks
- Coverage: Covergroups placed anywhere
- Checking: Custom classes

UVM

- Stimulus: Sequences
- Coverage: Same covergroups, written inside subscribers or monitors
- Checking: uvm_scoreboard (not much scaffolding provided)

SV testbenches vs. UVM testbenches

SV

- Test configuration: Custom variables, plusargs
- Communication: Mailboxes, FIFOs, function calls
- Test selection: Custom plusargs

UVM

- Test configuration: resource_db, factory
- Communication: Standard TLM ports/exports/FIFO
- Test selection: +UVM_TESTNAME (run_test())

SV testbenches vs. UVM testbenches

SV

- Virtual interface handling: Pass handles when creating classes
- Driver/Monitor code: Custom tasks (e.g. run())
- Create objects: new()

UVM

- Virtual interface handling: resource_db set() and get()/read_by_name()
- Driver/Monitor code: run_phase()
- Create objects: type_id::create()

SV testbenches vs. UVM testbenches

SV

- Reporting: $display, $error, $info, $warning

UVM

- Reporting: UVM_INFO, UVM_WARNING, etc.

Setting/Getting Interface Handle – UVM Style

System Verilog

- Pass handles to each class from the testbench
  - Either using an assignment
  - Or using a constructor
- Pros: Type-checking
- Cons: Have to assign to each class/component

UVM

- Put the handle into the resource_db from the testbench
- Get the handle from the resource_db from inside the class world
- Pros: One set, many gets
- Cons: String-based. Less type-checking

UVM Code Base

- Download from Accellera website
  - Download UVM (Universal Verification Methodology) - Accellera Systems Initiative
- Available with the simulator
  - VCS: Add “–ntb_opts uvm” to the command line
- Class documentation/reference
  - verificationacademy.com/verification-methodology-reference/uvm/docs_1.2/html/index.html

More resources

- UVM User guide
  - From Accellera website: https://accellera.org/images/downloads/standards/uvm/uvm_users_guide_1.2.pdf
- IEEE 1800.2-2020
  - IEEE SA - IEEE 1800.2-2020

More resources

- Verification Academy: Tools and training for functional verification.
- Synopsys training: https://training.synopsys.com/learn/courses/2937/language-systemverilog-verification-using-uvm

Complex/Advanced topics

- Factory overrides
- TLM put, get, blocking/non-blocking, etc
- Pre- and post- phases
- Run phase split into reset, main, shutdown, etc.
- Using UVM_ALL_ON related options
- Virtual sequence, virtual sequencer
- Sequencer-Driver handshake
- Sequence arbitration, RAL, Callbacks, …
- Simulation model slides (confusing)

Code walkthrough and execution

VS Code and Apporto

- easier_uvm.sv and easier_uvm_2.sv
- serialalu_uvm